

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №4

Выполнил:

Алексеев Андрей

Группа: К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

В рамках домашней работы требовалось спроектировать разбиение монолитного backend-приложения на микросервисы по принципу database-per-service, описать взаимодействие сервисов, выделить отдельные базы данных, подготовить межсервисные API-контракты и сформулировать конкретный план перехода от монолита к микросервисной архитектуре.

Ход работы

В качестве исходной системы рассматривается backend сервиса поиска работы, уже реализованный в lab1. В монолите присутствуют следующие прикладные группы endpoint'ов:

- auth;
- users;
- roles;
- companies;
- employer-profiles;
- industries;
- experience-levels;

- skills;
- resumes;
- vacancies;
- vacancy-assignments;
- applications;
- vacancies/favorites;
- vacancies/history.

При проектировании микросервисной архитектуры было важно не дробить приложение на слишком большое количество мелких сервисов, которые только увеличат сетевую связанность и административные издержки. Поэтому в качестве целевого варианта были выбраны четыре основных доменных сервиса. Отдельный микросервис только для поиска вакансий было решено не вводить, чтобы не разрывать один и тот же домен вакансий на спорный write/read split на уровне внешнего API.

1. identity-service

Сервис отвечает за:

- регистрацию и вход;
- учетные записи пользователей;
- роли;
- выпуск JWT;
- административные операции над ролями.

На него переносятся реальные endpoint'ы lab1:

- POST /auth/register
- POST /auth/login
- GET /users/me
- PATCH /users/me
- POST /users/{userId}/roles
- GET /roles
- POST /roles

2. recruitment-service

Сервис отвечает за:

- компании;
- профили работодателей;
- словари industry, experience_level, skill;
- вакансии;
- назначения работодателей на вакансии.

На него переносятся реальные endpoint'ы lab1:

- POST /companies
- GET /companies/{companyId}
- PATCH /companies/{companyId}

- POST /employer-profiles
- GET /employer-profiles/me
- PATCH /employer-profiles/me
- GET /industries
- POST /industries
- PATCH /industries/{industryId}
- GET /experience-levels
- POST /experience-levels
- PATCH /experience-levels/{experienceLevelId}
- GET /skills
- POST /skills
- PATCH /skills/{skillId}
- GET /vacancies
- POST /vacancies
- GET /vacancies/my
- GET /vacancies/{vacancyId}
- PATCH /vacancies/{vacancyId}
- DELETE /vacancies/{vacancyId}
- GET /vacancies/{vacancyId}/assignments

- POST /vacancies/{vacancyId}/assignments
- PATCH /vacancy-assignments/{assignmentId}
- DELETE /vacancy-assignments/{assignmentId}

Отдельный сервис для словарей было решено не вводить. В данной предметной области справочники слишком тесно связаны с вакансиями и резюме, а их объем и нагрузка недостаточны, чтобы оправдывать отдельный микросервис. Гораздо логичнее оставить их в рамках сервиса найма и публикации вакансий.

3. resume-service

Сервис отвечает за:

- резюме;
- образование;
- опыт работы;
- навыки резюме.

На него переносятся реальные endpoint'ы lab1:

- GET /resumes
- POST /resumes
- GET /resumes/{resumeId}
- PATCH /resumes/{resumeId}
- DELETE /resumes/{resumeId}

4. interaction-service

Сервис отвечает за:

- отклики;
- избранные вакансии;
- историю просмотров вакансий.

На него переносятся реальные endpoint'ы lab1:

- POST /vacancies/{vacancyId}/applications
- GET /vacancies/{vacancyId}/applications
- GET /applications/my
- PATCH /applications/{applicationId}/status
- GET /vacancies/favorites
- POST /vacancies/favorites/{vacancyId}
- DELETE /vacancies/favorites/{vacancyId}
- GET /vacancies/history
- POST /vacancies/history/{vacancyId}

Разделение базы данных

В целевой архитектуре используется четыре отдельных БД.

identity_db

- user
- role
- user_role

recruitment_db

- company
- employer_profile
- industry
- experience_level
- skill
- vacancy
- vacancy_skill
- vacancy_requirement
- vacancy_responsibility
- vacancy_benefit
- vacancy_assignment

resume_db

- resume
- resume_education
- resume_work_experience

- resume_skill

interaction_db

- application
- favorite_vacancy
- vacancy_view

Роль Kafka в архитектуре

В данной системе Kafka имеет смысл не “для галочки”, а для конкретных задач:

- обновление внутренних проекций поиска и кэшей внутри recruitment-service;
- обновление денормализованных read-моделей без постоянных синхронных запросов;
- сбор счетчиков просмотров, избранного и откликов для аналитики и ранжирования;
- ослабление связности между transactional-сервисами и аналитическими/поисковыми проекциями.

Предлагаемые события:

- vacancy.created
- vacancy.updated

- vacancy.deleted
- vacancy.published
- vacancy.assignment.changed
- company.updated
- dictionary.updated
- vacancy.viewed
- vacancy.favorited
- application.created
- application.status.changed

Основной потребитель этих событий — внутренняя поисковая и аналитическая проекция вакансий, принадлежащая recruitment-service или выделяемая позже как технический read-model контур без внешнего публичного API. За счет этого не приходится разрывать внешние endpoint'ы GET /vacancies и GET /vacancies/{vacancyId} между двумя сервисами: они остаются в recruitment-service, а Kafka используется только для внутренних read-моделей.

Синхронные взаимодействия между сервисами

Даже при наличии Kafka часть проверок должна оставаться синхронной.

- recruitment-service -> identity-service — проверка пользователя и его ролей при создании компаний, employer profile и вакансий.
- resume-service -> identity-service — проверка, что текущий пользователь действительно соискатель и владелец резюме.
- interaction-service -> resume-service — проверка существования резюме и его принадлежности пользователю.
- interaction-service -> recruitment-service — проверка публикационного статуса вакансии и прав работодателя на чтение и обработку откликов.

Связь архитектуры с реальными ручками lab1

Важный момент проектирования состоял в том, чтобы архитектура покрывала не “абстрактные сущности”, а реальные сценарии текущего приложения.

Примеры:

- создание вакансии уходит в recruitment-service;
- получение списка моих вакансий работодателя тоже уходит в recruitment-service;
- публичный поиск вакансий GET /vacancies и публичная карточка GET /vacancies/{vacancyId} тоже остаются в recruitment-service;
- создание резюме уходит в resume-service;
- отклики, избранное и история просмотров уходят в interaction-service;

- авторизация, роли, GET /users/me и PATCH /users/me остаются в identity-service.

Таким образом, разбиение строится не только вокруг таблиц, но и вокруг реально существующих HTTP-контрактов монолита.

Межсервисные API

Для синхронного межсервисного взаимодействия был подготовлен отдельный внутренний OpenAPI-контракт `openapi.yaml`. Важно, что этот файл описывает именно внутренние HTTP-контракты между микросервисами, а не весь публичный REST API монолита. В нем описаны:

- внутренние endpoint'ы identity-service для получения пользователя, ролей и проверки роли пользователя;
- внутренние endpoint'ы recruitment-service для employer profile, компании, справочников, публикационного статуса вакансии, interaction context и HR access;
- внутренние endpoint'ы resume-service для получения краткого контекста резюме и проверки владения резюме;
- внутренний endpoint interaction-service для чтения контекста отклика по applicationId.

Таким образом, внешний разрез по сервисам определяется реальными публичными ручками `lab1`, а `openapi.yaml` фиксирует только те синхронные вызовы, которые реально нужны сервисам после разделения монолита.

План миграции

Переход предлагается выполнять поэтапно.

1. Сначала выделить identity-service, поскольку auth и роли являются естественной границей.
2. Затем выделить recruitment-service, сохранив словари внутри него.
3. После этого выделить resume-service.
4. Затем выделить interaction-service.
5. После стабилизации сервисов добавить Kafka-проекции для поиска, аналитики и ранжирования вакансий, не меняя внешнюю границу recruitment-service.

Такой порядок уменьшает риск, потому что сначала фиксируются доменные границы, а уже потом добавляются event-driven read-модели.

Результат

В рамках домашней работы подготовлены следующие артефакты:

- отчет REPORT.md;
- архитектурное описание MICROSERVICE_ARCHITECTURE.md;
- внутренняя спецификация межсервисных endpoint'ов openapi.yaml.

Вывод

В результате был подготовлен более практичный вариант микросервисной архитектуры для текущего backend-приложения. Вместо избыточного дробления на слишком мелкие сервисы выбраны четыре основных доменных сервиса, а публичные ручки lab1 явно разложены по этим сервисам без потери текущего API-контракта. Все vacancy endpoint'ы, включая GET /vacancies и GET /vacancies/{vacancyId}, сохранены в одном recruitment-service; словари также оставлены внутри него, а Kafka используется там, где она действительно дает ценность: для построения внутренних поисковых и аналитических проекций, обновления денормализованных read-моделей и уменьшения синхронной связанности между сервисами.